# Evolução

Na Primeira versão do projeto tinhamos feito um aplicativo de interface de terminal que recebia um prompt e mostrava todas as perguntas e respostas em estruturas de mensagems de um chat de bate papo, com os criterios mais técnicos sendo um sistema de mémoria BufferMemory, que informava o historico completo para a ia, dois tipos de provedores possíveis para serem utilizados, ollama e gemini, nos prompts do sistema, cada system prompt tinha seu próprio arquivo contendo seu conteúdo integralmente.

Agora, na segunda versão, com a implementação do langchain, paramos de suportar dois provedores, focando integralmente em gemini, substituimos o BufferMemory por um TokenBufferMemory, assim tendo um controle maior sobre o gasto de tokens, e dividimos os system prompts, agora cada system prompt é separado em tópicos em arquivos diferentes, que são juntados em um único system prompt pelo ChatPromptTemplate do langchain. No restante, a interface visual continuou a mesma.

# Refatoração

Para Implementar e seguir o LCEL no projeto não foi muito dificil, já haviamos começado o projeto seguindo uma estrutura de adapters, então invés de nos comunicarmos diretamente com o gemini ou ollama, tinhamos uma classe generica chamada AiAdapter, que era implementada em classes como GeminiAdapter e ollamaAdapter, bastando alternar entre elas para mudar a ai, então para integrar o langchian criamos o LangChainAdapter.

Mas isso não quer dizer que não mudamos nada, enquanto as outros adapters, GeminiAdapter e OllamaAdapter, usavam fstring para implementar textos com parametros, o langchain faz isso por ChatPromptTemplate, então mudamos a estrutura para que fosse possível passar um dicionario de parametros para o ChatPromptTemplate.

Fora os aspectos que tangencia unicamente o pipeline da ia, mudamos também a estrutura dos system prompts, o que era antes um único arquivo para cada system prompt, agora, devido aos sistema de parametros do ChatPromptTemplate, foi refaturado para ter apenas um arquivo de system prompt, que severia de uma base gênerica, que depois recebe conteúdo extra por injeção pelos parâmetros, o que faz ser necessário modificar p paramêtro de system prompt que o aiAdapter recebe.

Fora do AiAdapter, o AiEngine, a parte responsável por receber o prompt do usuário e passar-lo pelo pipeline do sistema, sofreu pequenas modificações, com uma delas sendo não receber mais o historico do sistema, que agora seria administrado por uma classe de mémoria externa que estaria presente no AiEngine por composição, assim a classe externa poderia implementar qualquer regra de mémoria, sendo a escolhida atualmente o tokenBufferMemory

# Comparativo

## Sprint 2

| Pergunta | Acuracia | Estado |
|---|---|---|
| Quais carregadores estão disponíveis neste momento? | 0.75 | adequada |
| Minha recarga foi interrompida, o que aconteceu? | 0.8 | adequada |
| Como iniciar uma sessão de carregamento? | 0.6 | parcialmente adequada |
| Meu pagamento foi feito mas o carregador não funciona? | 0.8 | adaquada |
| O carregador não reconheceu meu veículo, o que eu faço? | 0.5 | parcialmente adequada |

## Sprint 3

| Pergunta | Acuracia | Estado |
|---|---|---|
| Quais carregadores estão disponíveis neste momento? | 0.75 | adequada |
| Minha recarga foi interrompida, o que aconteceu? | 0.6 | parcilamente adequada |
| Como iniciar uma sessão de carregamento? | 1 | adequada |
| Meu pagamento foi feito mas o carregador não funciona? | 0.8 | adaquada |
| O carregador não reconheceu meu veículo, o que eu faço? | 0.75 | adequada |